

Contents

Coding Agent — a minimal closed-loop control system (ch15 / §17)	1
Two policies, one seam	1
The §17 experiment	1
Layout (derived from SPEC.md §3–§11)	1
Development setup	2

Coding Agent — a minimal closed-loop control system (ch15 / §17)

A **minimal coding agent**: an LLM embedded inside a deterministic runtime that provides context, tools, state, permissions, execution, and verification. Its cycle is **observe** → **reason** → **act** → **verify** → **feedback** → **repeat** (curriculum §1/§18.2). It instantiates the chapter’s thesis —

*The system successfully navigated a software environment to a verified state. — by turning uncertain model output into measurable feedback via an in-loop **verifier**.*

The design is the deterministic-vs-probabilistic split carried forward from week 1: a **deterministic core** (sandbox, control loop, context manager, tool controller, permission gate, verifier, instrumenter, reporter — all LLM- and network-free) and a single **probabilistic** boundary, the model, reached *only* through the Policy interface.

Two policies, one seam

- **MockPolicy** — a deterministic offline double (a scripted or rule-driven action sequence). The *entire* automated suite runs over it with **no Ollama and no sockets**, so artifacts are **byte-identical** on the mock path (R-13 / I-002 / K-06).
- **OllamaPolicy** — opt-in, real qwen3.8 over <http://localhost:11434>. Isolated behind Policy (R-15 / I-009); on the default install it isn’t even present. A missing daemon is resolved by the model-availability taxonomy **DEGRADED MOCK** / **PULL_REQUIRED** / **RUN_REAL** (R-14 / E-11 / E-12) so a degraded run is never ambiguous in the trajectory.

The §17 experiment

The core acceptance activity is a **failure-injection** over a pinned fixture (`fixtures/parse-config/`): inject a *canonical defect* into `parse_config`, run the closed loop, and observe that the agent **detects** the FAILED verdict, **diagnoses** the cause, **repairs** it, and the number of iterations to VERIFIED is a *reproducible constant* (3) — not “whatever the model did today” (R-11 / I-013). Every run is instrumented (§17): `iteration`, `tool_calls`, `tokens`, `files_read`, `files_modified`, `tests_executed`, `test_results`, `errors`, `time_ms`, `final_outcome`.

Layout (derived from SPEC.md §3–§11)

```
src/coding_agent/  
task.py           # C-01 Task input  
policy.py         # C-02 Policy / MockPolicy / OllamaPolicy + resolve() taxonomy (R-14)  
permissions.py    # C-04 authorize() - the gate OUTSIDE the model (R-05 / I-008)  
tools.py          # C-03 ToolController + TOOL_SET, sandbox-bound (I-003)  
verifier.py       # C-05 VerifySpec / Verdict - closes the loop (R-06 / I-007)  
instrument.py     # C-06 trajectory iteration row + K-07 deterministic surrogates  
context.py        # R-03 context engineering + R-10 compaction (char budget, K-05/K-09)  
sandbox.py        # C-08-bis isolated, ephemeral sandbox + lifecycle (I-003 / I-011)  
control_loop.py   # R-01/R-08 the observe→...→feedback FSM + stopping conditions  
experiment.py     # R-11 the §17 detect/diagnose/repair experiment  
report.py         # C-06/C-07/R-16/R-17 write + schema-gate trajectory/experiment
```

```
cli.py          # R-16 the `agent` CLI: run / experiment / inspect / compare
fixtures/parse-config/  # the C-09 pinned unit under test + its verifier
schemas/        # C-06 trajectory.json + C-07 experiment.json (R-17 gate)
tests/          # SPEC §9 suite: T-01..T-13 (fully offline)
```

Development setup

```
uv sync          # default install: MOCK-ONLY, no model/network client
uv run pytest    # the SPEC §9 suite - fully offline, no Ollama, no sockets
uv run agent --help  # the CLI (R-16)
```

```
# opt in to the real Ollama policy (adds httpx; still never required by the suite):
uv pip install -e '[agents-real]'
```

The authoritative description of behavior, contracts, invariants, edge cases, and acceptance criteria is **SPEC.md**. Implementation is derived from it; see **SPEC.md §11 Traceability** for the id → module mapping.